

What are the main advantages of using CBVs?

مدیریت ساده تر متد های http، قابلیت استفاده مجدد و ارث بری، جدا شدن منطق نمایش و داده، قابلیت توسعه و سفارش سازی بالاتر

What are the main advantages of using CBVs?

: listview

این ویو برای نمایش لیستی از اشیاء استفاده میشود. وقتی از این ویو استفاده میکنیم معمولاً مشخص میکنیم که model چه باشد یا کوئری ست را مشخص میکنیم و سپس تمپلیت را پاس میدهیم. این ویو همچنین قابلیت pagination را داراست.

: detailview

برای نمایش جزئیات یک شی واحد از مدل استفاده میشود. نیاز دارد که pid یا slug شی را از url دریافت کند تا شی مورد نظر را پیدا کند.

: createview

این ویو برای ایجاد یک شی جدید از مدل بکار می رود. زمانی که Get می آید فرم مربوط به مدل نمایش داده می شود و هنگامی که Post می آید داده های فرم اعتبار سنجی شده و اگر معتبر بودند شی جدید ذخیره می شود.

: updateview

برای ویرایش یک شی موجود استفاده می شود. فرم مقادیر موجود در شی را به صورت پیش فرض میگیرد تا کاربر بتواند آنها را تغییر داد.

: deleteview

برای حذف شی از مدل استفاده می شود.

What is the purpose of get_queryset() and form_valid() methods, and how can you override them?

:get_queryset

این متد مسئول بازگرداندن مجموعه اشیا است که ویو قرار است با آنها کار کند. اگر به صورت پیش فرض در مدل query set را مشخص کرده باشیم از تعریف خودمان پیروی میکند در غیر این صورت objects.all را برمیگرداند.

:form_valid

این متد مشخص میکند که وقتی داده های فرم معتبر بودند، جنگو باید چه کاری انجام دهد. این متد به صورت پیش فرض بعد از اعتبار سنجی فرم توسط جنگو، فراخوانی می شود و فرم را ذخیره میکند. اگر بخواهیم بعد از ذخیره سازی کاری اضافه انجام دهیم مثل تنظیم فیلدی خاص یا ثبت لاگ یا هر کار جانبی دیگر، میتوانیم در form_valid قرار دهیم.

How does `form.is_valid()` work, and when is `form_valid()` triggered?

: `form.is_valid`

یک متد از کلاس `Form` یا `ModelForm` است که جنگو برای بررسی اعتبار سنجی داده های ارسالی از کاربر استفاده میکند. جنگو تمام فیلد های فرم را بررسی میکند تا داده ها موجود باشند و با نوع مورد انتظار سازگار باشند، داده هارا با محدودیت های مدل هم مقایسه میکند. متد های `clean` و `clean_fieldName` را نیز فراخوانی میکند تا اعتبارسنجی سفارشی اجرا شود. اگر هیچ خطایی پیدا نشود `form.clean_data` ساخته می شود و مقدار `true` برمیگرداند در غیر اینصورت `form.errors` پر می شود و `false` برمیگرداند. زمانی که از CBVs استفاده میکنیم، جنگو در متد `post` اجرا می شود و اگر نتیجه `true` بود به طور خودکار `form_valid` را صدا میزند.

How do you connect CBVs to URLs using the `as_view()` method?

چون ویو ها کلاس هستند و کلاس به تنهایی قابل فراخوانی نیست، باید آن را به یک callable view تبدیل کنیم. این کار با متد `as_view` انجام می شود.

How can you perform field-level validation using the `clean_<fieldname>()` method?

نام متد باید به صورت `clean_fieldname` باشد که `fieldname` همان نام فیلد در فرم است. این متد باید مقدار نهایی فیلد را برگرداند یا در صورت نامعتبر بودن یک `validation error` پرت میکند.

```
def clean_age(self):
    age = self.cleaned_data.get('age')
    if age < ۱۸:
        raise ValidationError("age should be greater than ۱۸")
    return age
```

What is the difference between `clean()` (form-level) and `clean_<fieldname>()` (field-level) validation?

اعتبار سنجی سطح فیلد:
مربوط به یک فیلد خاص است
فقط مقدار همان فیلد را بررسی میکند
اگر مقدار نامعتبر باشد validation error پرت میکند در غیر اینصورت مقدار معتبر را برمیگرداند و در `clean data` ذخیره میکند.

اعتبار سنجی سطح فرم:
مربوط به کل فرم و همه فیلدها است
برای اعتبار سنجی بین فیلدها یا اعمال قوانین پیچیده استفاده می شود.

How can you handle validation errors and display them in templates?

اعتبارسنجی فرم با `form.is_valid` انجام می شود.
خطاها در `form.errors` و `form.<field>.errors` ذخیره می شوند.
قالب ها می توانند با `{{ form.field.errors }}` و `{{ form.non_field_errors }}` خطاها را نمایش دهند.
در `CBV`، متد `form_invalid` این کار را به صورت خودکار انجام می دهد.

How can you use Django's messages framework to give users feedback after create, update, or delete actions?

عملیات ایجاد، ویرایش یا حذف در `CBV` انجام می شود.
در `form_valid` یا `delete` پیام مناسب با `messages.success` اضافه می کنیم.
بعد از ریدایرکت یا بارگذاری دوباره صفحه، پیام ها در قالب با حلقه `for message in messages` نمایش داده می شوند.

Search about CSRF tokens. Why they are important?

CSRF مخفف Cross-Site Request Forgery است.

این نوع حمله زمانی اتفاق می‌افتد که یک کاربر معتبر ناخواسته عملیاتی را روی یک وبسایت انجام دهد که مهاجم قصد دارد انجام شود.
مثال: کاربری که در حساب بانکی خودش وارد شده، بدون اطلاعش یک درخواست POST ارسال می‌کند و پول منتقل می‌شود.
هدف CSRF جلوگیری از این نوع درخواست‌های جعلی است.

نقش CSRF token

جنگو برای محافظت از فرم‌ها و درخواست‌های POST از CSRF token استفاده می‌کند.
CSRF token یک رشته منحصر به فرد و تصادفی است که برای هر نشست (session) یا فرم ایجاد می‌شود.
وقتی فرم با POST ارسال می‌شود، جنگو بررسی می‌کند که این token با token ذخیره شده در session مطابقت دارد.
اگر token معتبر نباشد : درخواست رد می‌شود و ۴۰۳ Forbidden برمی‌گردد.